

《Linux基础与应用》课程大作业

学生姓名：王志蕴

学号：0242633

摘要

本次大作业旨在构建一个完整的 LAMP (Linux + Apache + MySQL + Python(Django)) 环境。我在 Ubuntu 系统上配置了用户与网络安全，手动部署了基于 Django 和 YOLOv8 的“智能图库”Web 应用，实现了图片的上传、AI 自动识别分类及管理功能。同时，编写了综合 Shell 脚本实现了服务器资源监控、日志分析、服务状态报警及自动化灾备，模拟了真实的生产运维场景。其间出现了好多问题,最主要是在Django项目部署到apache上出现的环境依赖问题,但都一一解决。

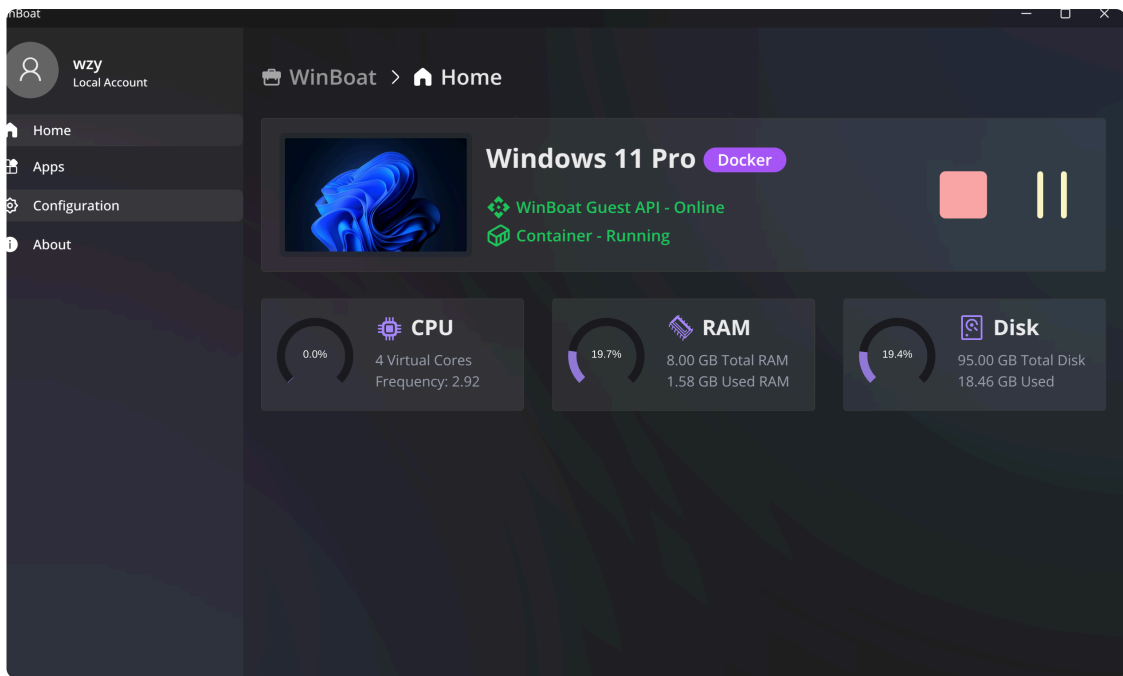
-----🚀 王志蕴的Linux大作业

核心目标： 搞定一个完整的 LAMP (Linux + Apache + MySQL + Python) 环境，外加一个带 AI 功能的智能图库应用和一套自动化运维脚本。

一、系统搭建与安全基础

- 🖥️ 系统环境： 我已经在用了Ubuntu Linux作为主要使用系统，IP 地址是 192.168.210.32。后面用docker部署了一个开源项目winboat🔗 <https://github.com/TibixDev/winboat>的window 11 pro系统,主端口为47300

- ```
ver, for secure access from remote machines
> ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
 link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
 inet 127.0.0.1/8 scope host lo
 valid_lft forever preferred_lft forever
 inet6 ::1/128 scope host noprefixroute
 valid_lft forever preferred_lft forever
2: enp4s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
 link/ether b0:25:aa:66:e0:0b brd ff:ff:ff:ff:ff:ff
3: wlo1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
 link/ether e4:60:17:9a:0a:24 brd ff:ff:ff:ff:ff:ff
 altname wlp0s20f3
 inet 192.168.210.32/24 brd 192.168.210.255 scope global dynamic noprefixroute wlo1
 valid_lft 2699sec preferred_lft 2699sec
 inet6 240a:42b6:500:1f58:644:9566:7a7:a4a6/64 scope global temporary dynamic
 valid_lft 7178sec preferred_lft 7178sec
 inet6 240a:42b6:500:1f58:9b71:936:27:d95a/64 scope global dynamic mngtmpaddr noprefixroute
 valid_lft 7178sec preferred_lft 7178sec
 inet6 fe80::9e7d:cdde:b471:128e/64 scope link noprefixroute
 valid_lft forever preferred_lft forever
4: zt5bhydowu: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 2800 qdisc fq_codel state UNKNOWN group default qlen 1000
 link/ether ba:81:90:c9:2f:87 brd ff:ff:ff:ff:ff:ff
 inet 10.244.119.253/16 brd 10.244.255.255 scope global zt5bhydowu
 valid_lft forever preferred_lft forever
 inet6 fe80::b881:90ff:fe09:2f87/64 scope link
```



- 用户管理：

- 创建了主用户 wzy (我的拼音名)。
- 创建了用户组 jxufe 和两个辅助用户 user01, user02，并把他们都拉进组里。
- 创建名为 `jxufe` 的用户组，以及 `user01` 和 `user02` 两个用户,wzy 有 `sudo` 权限。

```
创建用户组
sudo groupadd jxufe

创建其他用户
sudo adduser user01
sudo adduser user02
```

```

> grep jxufe /etc/group (base)
jxufe:x:1002:wang,user01,user02,wzy
> sudo -l (base)
匹配 wzy-ubuntu 上 wzy 的默认条目:
 env_reset, mail_badpass,
 secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/b
in\:/sbin\:/bin\:/snap/bin,
 use_pty

wzy Runas 和命令特定的默认值:
 Defaults!/usr/lib/*/libexec/kf5/kdesu_stub !use_pty

用户 wzy 可以在 wzy-ubuntu 上运行以下命令:
 (ALL : ALL) ALL

```

(base)

-  远程连接：装了 OpenSSH Server，能直接从 Windows 电脑 SSH 连上去操作

```

> dpkg -l | grep openssh-server (base)
ii openssh-server 1:9.6p1-3ubuntu13.14 amd64 secure shell (SSH) ser
ver, for secure access from remote machines

```



```
[wzy-ubuntu] ~
Microsoft Windows [版本 10.0.26100.4349]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\wzy>ssh wzy@192.168.210.32
wzy@192.168.210.32's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-37-generic x86_64)

 * Documentation: https://help.ubuntu.com
 * Management: https://landscape.canonical.com
 * Support: https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

35 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sat Dec 27 16:57:49 2025 from 172.18.0.2
./+oossssoo+/-
':+ssssssssssssssssss+:`
--+ssssssssssssssssssyyssss+--
.ossssssssssssssssssdMMMMNyssssso.
/ssssoosssssssshdmmNNmmyNMMMMHssssss/
+ssssssssshmydMMMMMMNdssssssssss+
/ssssoosssshNMMMyhhyyyhmNMMMMHssssss/
.ssssoosssdMMMMHssssssssshNMMMdssssss.
+ssssshhhyNMMNysssssssssssyNMMMyssssss+
ossyNMMMyMMHssssssssssssshmmhssssssso
ossyNMMMyMMHssssssssssssshmmhssssssso
+ssssshhhyNMMNysssssssssssyNMMMyssssss+
.ssssoosssdMMMMHssssssssshNMMMdssssss.
/ssssoosssshNMMMyhhyyyhdNMMMMHssssss/
+ssssssssshmydMMMMMMNdssssssssss+
/ssssoosssshdmmNNmmyNMMMMHssssss/
.ossssssssssssssssssdMMMMNyssssso.
--+ssssssssssssssssssyyssss+--
':+ssssssssssssssssss+:`
./+oossssoo+/-

wzy@wzy-ubuntu

OS: Ubuntu 24.04.3 LTS x86_64
Host: KuangshiG16 Series GM6PD0Q Standard
Kernel: 6.14.0-37-generic
Uptime: 5 hours, 31 mins
Packages: 2457 (dpkg), 18 (snap)
Shell: fish 3.7.0
Resolution: 1366x768
Terminal: /dev/pts/6
CPU: 13th Gen Intel i7-13620H (16) @ 4.700GHz
GPU: NVIDIA GeForce RTX 4060 Max-Q / Mobile
Memory: 18619MiB / 31823MiB

Welcome to fish, the friendly interactive shell
Type help for instructions on how to use fish
> |
```

## 二、Apache 网站服务

- 服务启动：用 apt 装了 Apache2，systemctl status apache2 一看，运行状态 active (running)!



```
sudo systemctl status apache2
apache2.service - The Apache HTTP Server
Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
Active: active (running) since Mon 2025-12-29 11:43:20 CST; 5h 17min ago
Docs: https://httpd.apache.org/docs/2.4/
Process: 17139 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
Main PID: 17143 (apache2)
Tasks: 73 (limit: 37834)
Memory: 393.8M (peak: 394.2M)
CPU: 3.388s
CGroup: /system.slice/apache2.service
├─17143 /usr/sbin/apache2 -k start
├─17182 /usr/sbin/apache2 -k start
├─17183 /usr/sbin/apache2 -k start
└─17185 /usr/sbin/apache2 -k start

12月 29 11:43:20 wzy-ubuntu systemd[1]: Starting apache2.service - The Apache HTTP Server...
12月 29 11:43:20 wzy-ubuntu apachectl[17142]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, >
12月 29 11:43:20 wzy-ubuntu systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-18/18 (END)
```

- 防火墙配置 (UFW):

- 通过 `sudo ufw enable` 启用防火墙，开了ssh(22),Apache,djaogo(8000),mysql(3306)这几个必要的端口。

- ```
> sudo ufw status
```

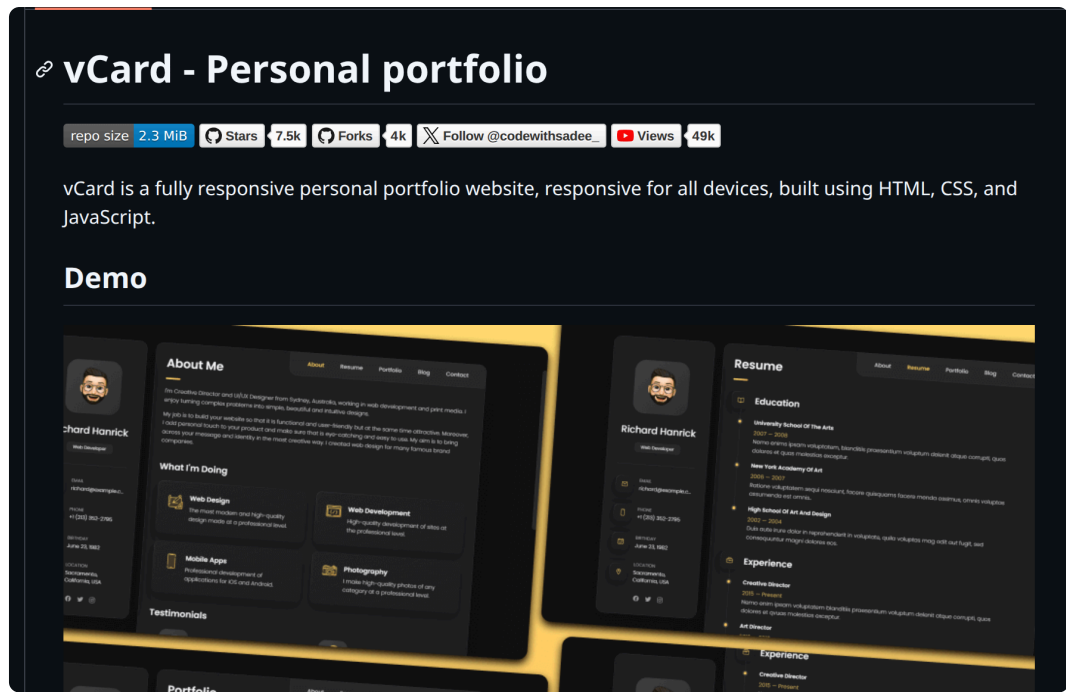
状态: 激活

至	动作	来自
-	-	-
3306/tcp	ALLOW	Anywhere
22/tcp	ALLOW	Anywhere
Apache	ALLOW	Anywhere
8000	ALLOW	Anywhere
3306/tcp (v6)	ALLOW	Anywhere (v6)
22/tcp (v6)	ALLOW	Anywhere (v6)
Apache (v6)	ALLOW	Anywhere (v6)
8000 (v6)	ALLOW	Anywhere (v6)

- 个人主页:

- 个人主页的模板来源于github中一个开源目:

<https://github.com/codewithsadee/vcard-personal-portfolio>



- 通过将写好的静态文件放到Apache中的/var/www/html中实现部署

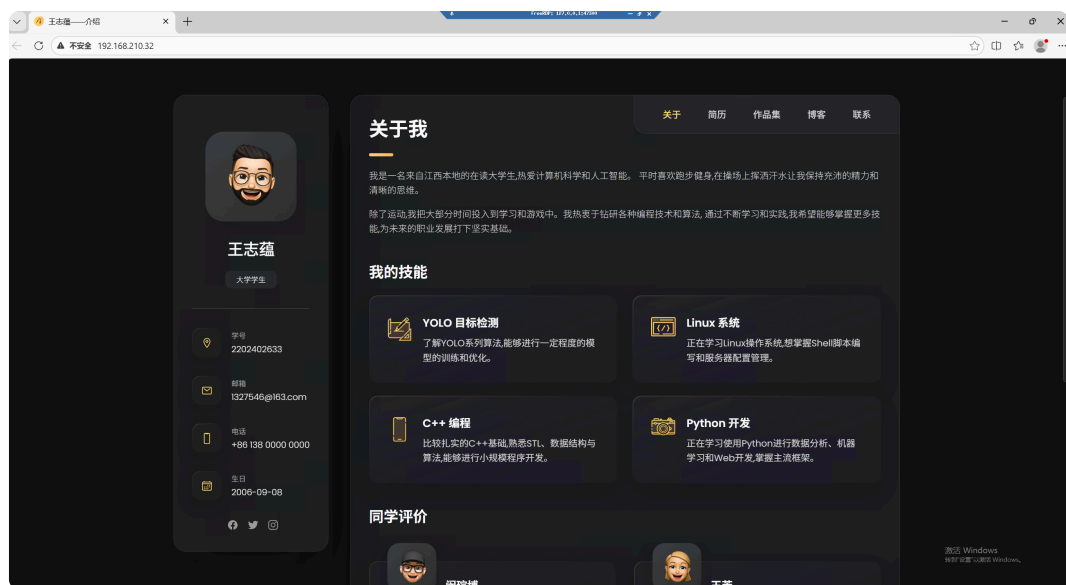
```

/vw/html

> cd /var/www/html/
> ls
assets/  index.html  index.txt  LICENSE  README.md  website-demo-image/
🔒 /var/www/html

```

- 设置了防火墙使得window虚拟机能在 IP 地址 <http://192.168.210.32> 上访问包含个人信息的静态主页，浏览器访问成功。最终呈现的效果如下,包含名字和学号:



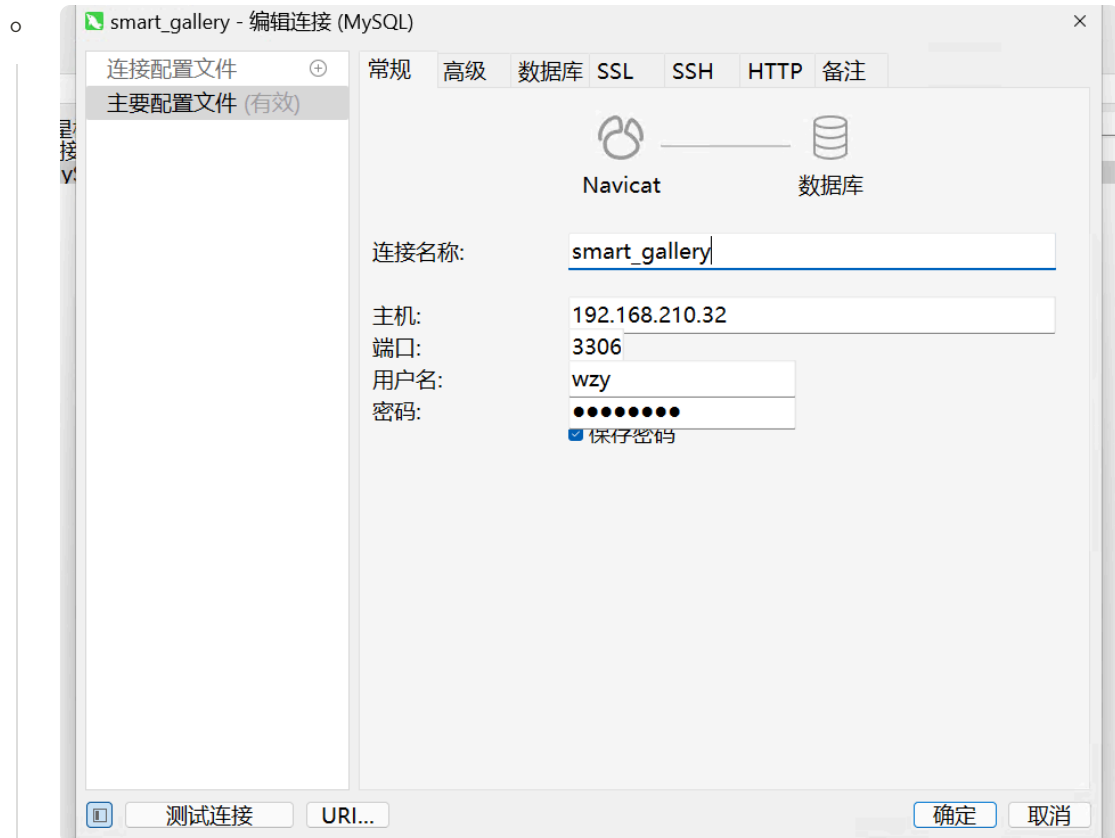
三、MySQL 数据库服务 (上数据库课时就安装过)

-  安装与配置：装好 MySQL Server，创建了linux_homework(展示信息),smart_gallery(配合项目) 数据库，还给用户 wzy 授权了。

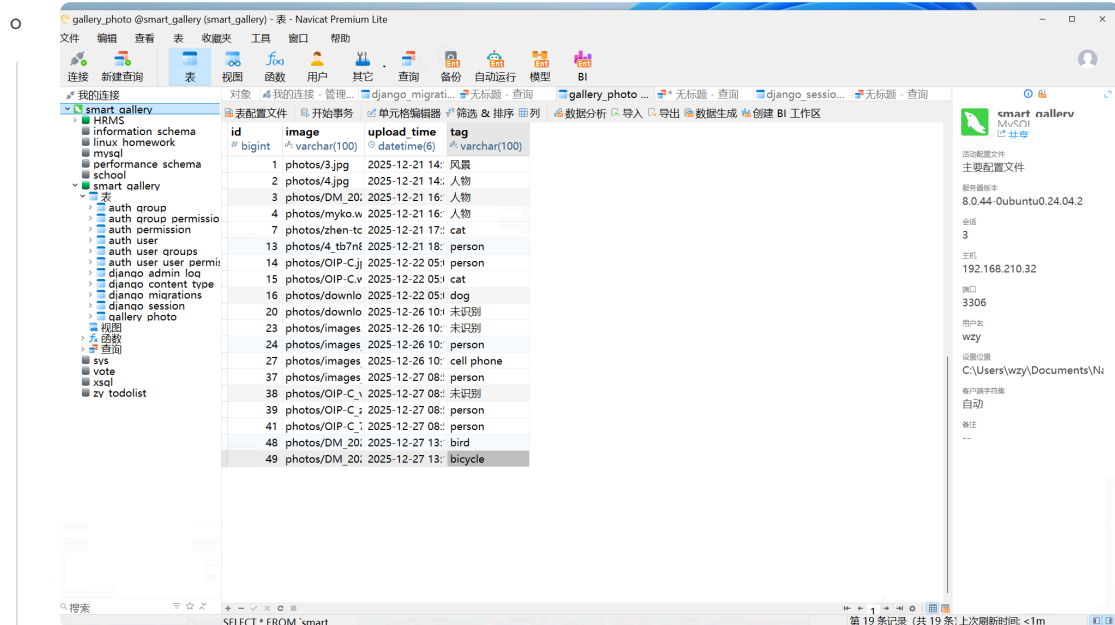
```
mysql> select * from student;
```

```
+-----+-----+-----+
| id      | name      | number      |
+-----+-----+-----+
| 242633   | 王志蕴    | 2202402633  |
+-----+-----+-----+
1 row in set (0.00 sec)
```

-  远程访问：
 - 前面已设置把防火墙打开,把 bind-address 改成 0.0.0.0，允许所有地址远程连接。



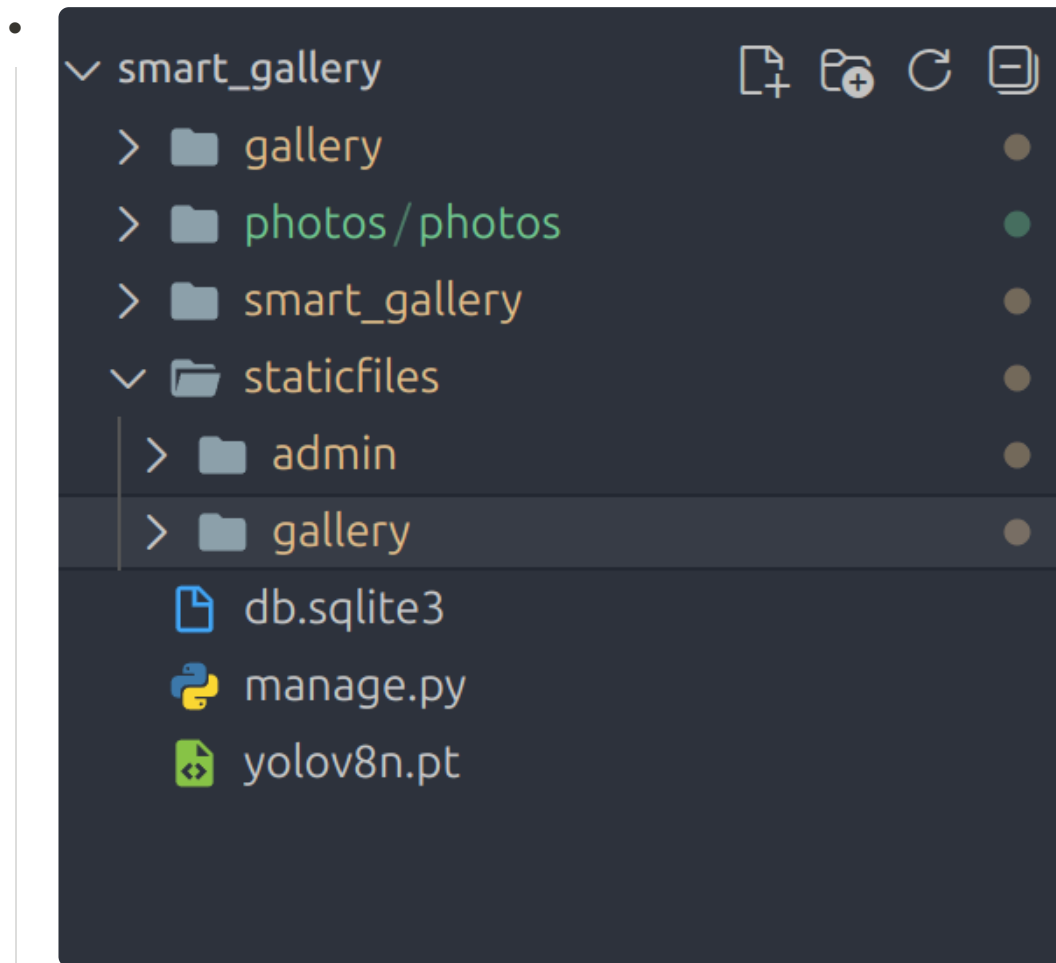
- o 用宿主机上的数据库工具navicat成功连上，并且能查到数据，远程操作无压力！



四、Django 智能图库应用部署 (花时间最长的一部分)

建立项目

- 建立项目 `django-admin startproject`
- 创建app `python manage.py (runserver/app)`



- conda下载yolo相关库
- 在setting.py文件中将默认的sqlite3改成mysql



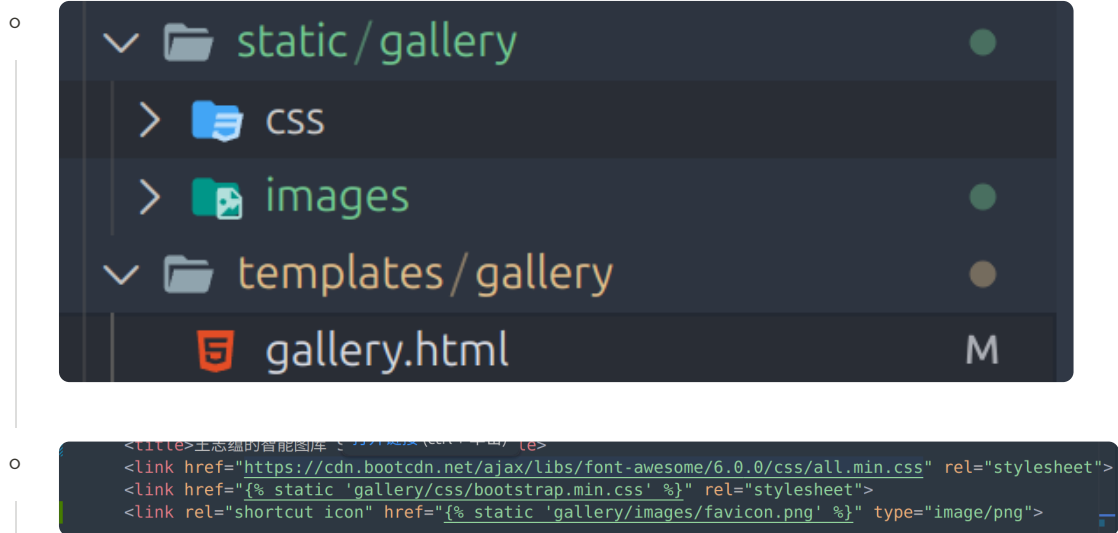
- 在models.py中定义类,运行 `python manage.py makemigrations`（生成迁移文件）,运行 `python manage.py migrate`,将自动将定义的类在mysql中建立对应的table

```
views.py gallery.html M favicon.jpg U favicon.png U settings.py M models.py X
gallery > models.py > Photo > __str__ >
1 from django.db import models
2
3 # Create your models here.
4 class Photo(models.Model):
5     #image tag upload time
6     image = models.ImageField(upload_to='photos/')
7     upload_time = models.DateTimeField(auto_now_add=True)
8     tag = models.CharField(max_length=100, blank=True, null=True)
9     def __str__(self):
10         return f"Photo {self.id} - {self.tag or 'No Tag'}"
```

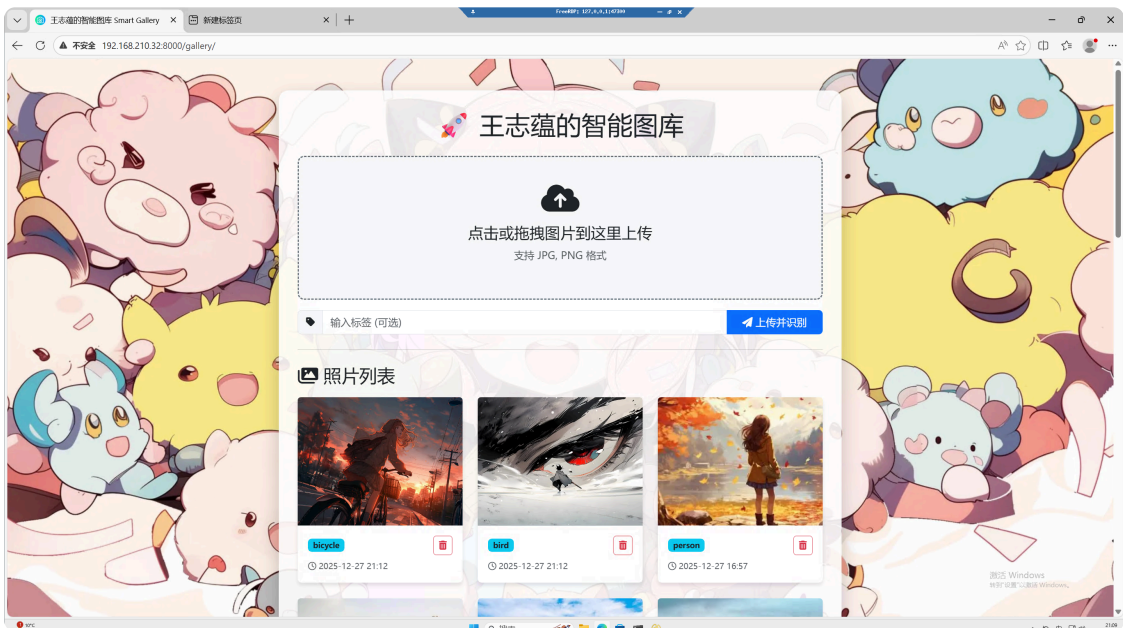
- 在views.py中实现了写标签,yolo自动打标签,删除图片的逻辑

```
smart_gallery - views.py
views.py X gallery.html M favicon.jpg U favicon.png U settings.py M
gallery > views.py > gallery_view >
1 from django.shortcuts import render, redirect, get_object_or_404 # <--
2 from .models import Photo
3 from ultralytics import YOLO
4 from django.conf import settings
5 import os
6
7 # 预加载模型
8 print("正在加载 AI 模型...")
9 model_path = os.path.join(settings.BASE_DIR, 'yolov8n.pt')
10 model = YOLO(model_path)
11
12 def gallery_view(request):
13     if request.method == 'POST':
14         # ===== 删除逻辑开始 =====
15         # 如果请求里包含 'delete_id', 说明用户点了删除按钮
16         if 'delete_id' in request.POST:
17             photo_id = request.POST.get('delete_id')
18             # 找到照片, 找不到会报 404 (比较安全)
19             photo = get_object_or_404(Photo, id=photo_id)
20
21             # 1. 先删硬盘上的文件
22             photo.image.delete(save=False)
23             # 2. 再删数据库里的记录
24             photo.delete()
25
26             # 删完刷新页面
27             return redirect('gallery:photo_list')
28         # ===== 删除逻辑结束 =====
29
30         # ===== 上传逻辑开始 (原来的代码) =====
31         tag = request.POST.get('tags', '')
32         image = request.FILES.get('image')
33         if image:
```

- 新建templates 放前端代码块, 加了一个中间层gallery, 新建static 放静态文件(一开始css是直接引用外部资源, 但是国外的源, win访问不了, 就下载了css和换国内源了)



- 用python manage.py runserver 直接运行的, win也能通过192.168.210.32:8000/gallery/查看的



- 至此, 项目基本完成

将django部署在apache中(环境配置搞的我头大)

本来我觉的django就已经可以了, 但作业要求用apache, 弄了好久

主要问题:

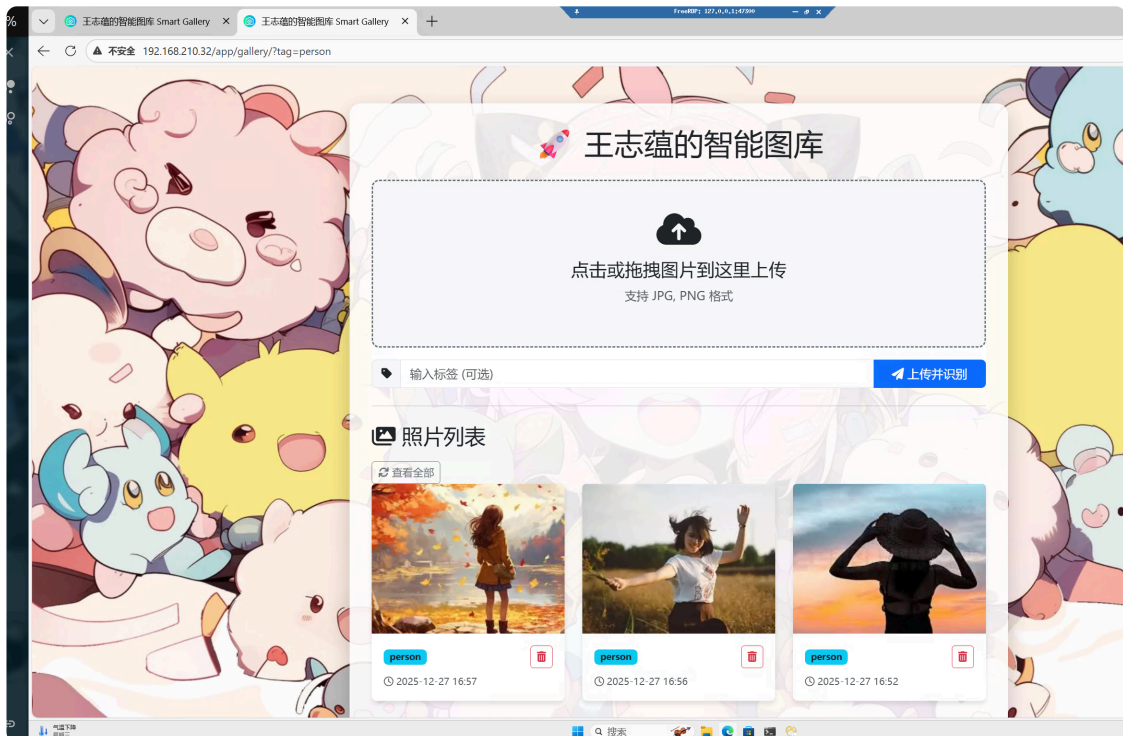
- 权限问题,路径问题,中文识别问题(就因为我的文件夹用了中文,通过export LANG='C.UTF-8'

export LC_ALL='C.UTF-8'解决),css 渲染不出来的问题,图片上传不了的问题

- **最大问题:**conda依赖问题 ,django库找不到
 - 我之前用的是conda python 3.14 但apache是python 3.12 ,只能conda降级
 - yolo死活报错,一直找不到问题,第二天才发现用apt install mod_wsgi默认用的是ubuntu 自带的python环境,和conda 中的不兼容

主要过程:

- 先安装mod_wsgi(不能用apt安装,导致我后面排查很久)
- 再在settings中处理静态文件
- 再用python manage.py collectstatic整合静态文件
- 编辑 Apache2配置 sudo nano /etc/apache2/sites-available/smart_gallery.conf
- 授予权限,最终终于成功了,可以通过 <http://192.168.210.32/app/gallery/> 访问



概述小结

-  项目概览 (Smart Gallery):
 1. 技术栈: Python 3.12, Django 5.2, MySQL, 还有迷你板的 YOLOv8 AI 识别!

2. 核心功能：图片上传、AI 自动识别物体并打标签、删除（数据库和文件一起删），支持拖拽上传。

- 🏗️ 生产级部署：没用 Django 自己的 runserver，直接上了工业级标配 Apache + mod_wsgi!

1. 环境隔离：用 Conda 虚拟环境 (others) 来管理 Python 依赖，环境干净不串线。
2. Apache 配置：写了 smart_gallery.conf 文件，把 WSGI 进程指向了 Conda 环境，并映射了静态文件 (/app/static) 和上传的媒体文件 (/media)。应用跑在 /app 路径下。

- 🧠 遇到的问题和解决办法：

1. 静态文件找不到？运行 collectstatic 统一收集，并在 Apache 里设置 Alias 映射，搞定！
2. 文件权限被拒绝？Apache 运行用户 www-data 没权限写上传目录。简单粗暴：sudo chmod -R 777 photos，解决权限问题！
3. 环境兼容性？Apache 默认模块和 Conda 环境不兼容。高分操作：在 Conda 环境里自己编译 mod_wsgi 专用模块并加载，完美兼容！

- ✨ 运行效果：访问 <http://192.168.210.32/app/gallery/>，可以看到带 AI 标签的图片列表、拖拽上传界面和删除功能，效果还行吧,识别的不是很准确,yolo8n的锅！

五、Shell 编程与自动化运维

- ⚙️ 运维脚本 (ops_manager.sh)：编写了一个带交互式菜单的超级运维脚本，一站式解决运维需求。

```
00% [Ctrl+L]
===== Linux 运维管理工具 =====
1. 📊 实时资源监控 (CPU/内存/磁盘)
2. 📈 Apache 日志流量分析
3. 🧹 日志备份与过期清理
4. 🔍 服务运行状态监控 (Httpd/MySQL)
5. 🚪 退出系统
=====
请输入选项 [1-5]:
```

- 实时监控：用 top, free, df 提取 CPU/内存/磁盘使用率，每 5 秒自动刷新。

```
o  [+] S
===== [系统资源监控] 2025-12-29 21:42:48 =====
CPU 使用率：12.2%
内存使用率：61.00%
磁盘使用率：40%
=====
```

- o 日志分析：统计当天总请求数，并分析出 Top 10 热门访问 URL (解决了日志日期格式的坑)。

```
o  [+] S
1. 📈 实时资源监控 (CPU/内存/磁盘)
2. 📊 Apache 日志流量分析
3. 🧹 日志备份与过期清理
4. 🚨 服务运行状态监控 (Httpd/MySQL)
5. 🚪 退出系统

=====
请输入选项 [1-5]: 2
📊 正在分析 Apache 日志...

-----
📅 分析日期：29/Dec/2025
📊 当日总请求数：173

-----
🏆 当日访问量 Top 10 URL:
    16 /app/gallery/
    4 /media/photos/OIP-C_z6hNYqa.webp
    4 /media/photos/OIP-C_vCTY6NR.webp
    4 /media/photos/OIP-C_7RlxfA.webp
    4 /media/photos/images_rzk4yLD.jpg
    4 /media/photos/images.jpg
    4 /media/photos/images_IBJdmqk.jpg
    4 /media/photos/images_i8Se0bc.jpg
    4 /media/photos/download_Zr8NjGy.jpg
    4 /media/photos/download.webp

-----
✅ 分析完成。

按回车键返回主菜单... [ ]
```

- 日志管理：自动打包备份 /var/log/apache2 下的日志，并清理 30 天前的旧日志。

```

===== Linux 运维管理工具 =====
1. 📊 实时资源监控 (CPU/内存/磁盘)
2. 📈 Apache 日志流量分析
3. 🧹 日志备份与过期清理
4. 🚦 服务运行状态监控 (Httpd/MySQL)
5. 🚪 退出系统
=====
请输入选项 [1-5]: 3
👉 开始执行日志维护任务...
✅ 创建备份目录: /tmp/apache_backups
📦 正在打包备份日志到 /tmp/apache_backups/apache_logs_20251229.tar.gz ...
tar: 从成员名中删除开头的 "/"
✅ 备份成功!
-----
🗑️ 正在检查并清理 30 天前的旧日志...
✅ 清理操作完成。

按回车键返回主菜单...

```

- 服务报警（高光时刻）：

- 实时监测 Apache2 和 MySQL 状态。
- 邮件功能：服务宕机，发送邮件,原本是用bash脚本写,但是没认证163邮件拒收,然后用bash连接 Python 脚本发邮件通知管理员，确保第一时间知道故障（还解决了 Linux mail 命令配置的麻烦）！

```

# 组合邮件正文
mail_body="警告！以下服务出现异常，请立即检查: \n\n${failed_services_text}\n检测时间: ${date}"

# 发送邮件（注意要把 mail_body 用双引号包起来）
python3 ./send_mail.py "$mail_body" "【严重】Linux服务聚合报警"

```

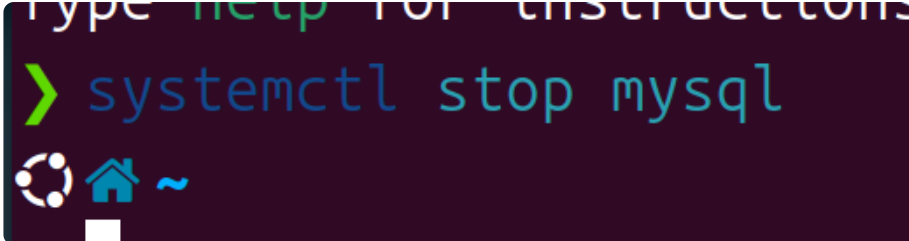
```

📧

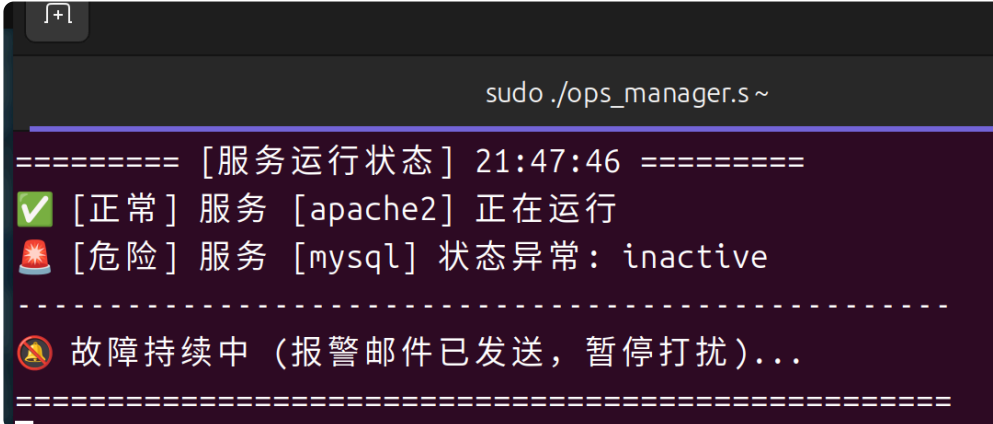
===== [服务运行状态] 21:47:00 =====
✅ [正常] 服务 [apache2] 正在运行
✅ [正常] 服务 [mysql] 正在运行
-----
=====

```

- 防骚扰：实现了故障期间只发一次报警的机制，防止邮件轰炸，人性化！

- 

```
type help for instructions
> systemctl stop mysql
```

- 

```
sudo ./ops_manager.s ~
===== [服务运行状态] 21:47:46 =====
[✓] [正常] 服务 [apache2] 正在运行
[🔥] [危险] 服务 [mysql] 状态异常: inactive
-----
[🚨] 故障持续中 (报警邮件已发送, 暂停打扰)...
=====
```

- 

【严重】Linux服务聚合报警 | 安全浏览模式

发件人: 我<13207967546@163.com> +

收件人: 我<13207967546@163.com> +

时 间: 2025年12月29日 21:47 (星期一)

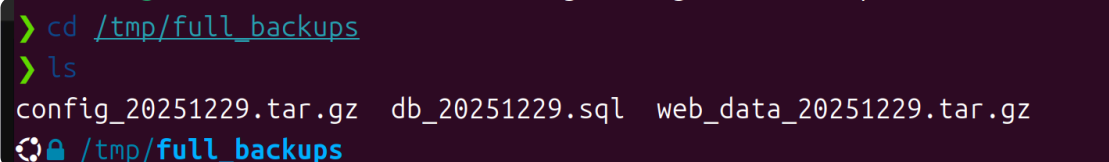
AI写信+润色, 轻松写出高转化率的电子邮件

警告! 以下服务出现异常, 请立即检查:

 - 服务: mysql (状态: deactivating)

检测时间: 2025年 12月 29日 星期一 21:47:30 CST

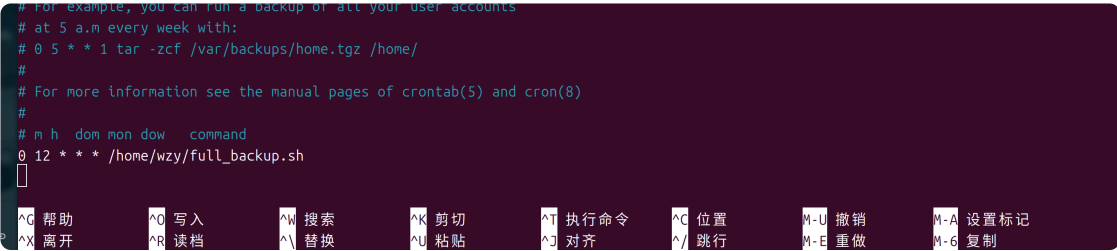
- 📅 自动化备份 (Crontab):
- 写了 full_backup.sh 脚本, 专门用来备份网站源码、配置和数据库 SQL。

- 

```
> cd /tmp/full_backups
> ls
config_20251229.tar.gz  db_20251229.sql  web_data_20251229.tar.gz
```

- 配置了 Crontab 定时任务: 0 12 *** /home/wzy/code/full_backup.sh, 每天凌晨 02:00 自

动全量备份，数据安全有保障！

- A terminal window with a dark purple background. It shows a crontab configuration for a backup script. The text includes: "# For example, you can run a backup of all your user accounts", "# at 5 a.m every week with:", "# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/", "#", "# For more information see the manual pages of crontab(5) and cron(8)", "#", "# m h dom mon dow command", "0 12 * * * /home/wzy/full_backup.sh". Below the terminal, there is a row of keyboard shortcuts: ^G 帮助, ^O 写入, ^W 搜索, ^K 剪切, ^T 执行命令, ^C 位置, M-U 撤销, M-A 设置标记, ^X 离开, ^R 读档, ^_ 替换, ^U 粘贴, ^J 对齐, ^/ 跳行, M-E 重做, M-6 复制.

六、总结

心得体会：

- 通过这次大作业，逐渐理解了 Linux 服务器的整套运维流程，从系统底层到上层应用的部署，再到自动化运维和灾备。
- 亲手实现了生产环境的“一条龙”服务：应用部署、实时监控、自动化报警和定时备份，把书本上的理论（比如用户管理、网络服务）真正用起来，变成一个跑起来的 Web 应用，这种理论与实践结合的感觉超有成就感。
- 深刻体会到开源技术的强大，特别是 YOLOv8 这种 AI 模型，让 Web 应用的功能瞬间升级！

能力提升：

- 虽然遇到了环境冲突、权限拒绝、路径映射等各种问题，但都通过分析日志和查资料解决了，解决问题的能力获得了极大的提升！
- 掌握了排查生产环境问题的关键技能，比如通过日志定位静态文件、通过权限设置解决写入问题，学会了如何“驯服”Apache。
- 学会了在复杂环境中（比如 Conda 虚拟环境）进行定制化配置和编译 mod_wsgi 专用模块，解决了系统兼容性，这个技术点非常硬核。
- 对 Shell 脚本的编写能力有了质的飞跃，不仅能实现交互式菜单、日志分析，还能实现服务宕机邮件报警的复杂逻辑。